

Reviewer Pack — Fourier L1 Norm Cubed-Polynomial Bound for Read-Twice BPs

Entry 90dea97bbe27 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 11:54:04 UTC

Fourier L1 Norm Cubed-Polynomial Bound for Read-Twice BPs

Entry ID: 90dea97bbe27 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 11:54:04 UTC

1. Conjecture

Field A (mathematical branch): Fourier analysis on the Boolean cube — spectral (Fourier- $\square^1$ / μ -) norm in the Kahn-Kalai-Linial / Bruck-Smolensky tradition **Field B** (complexity object): Read-twice oblivious branching programs (BP_READTWICE), Borodin-Razborov-Smolensky 1993 setting; size is total node count

Statement:

Let P be a read-twice oblivious branching program of size $s = \text{size}(P)$ computing $f : \{0,1\}^n \rightarrow \{-1,+1\}$. Then its Fourier $\square^1$ norm satisfies $\|\hat{f}\|_1 := \sum_{S \subseteq [n]} |\hat{f}(S)| \leq (s+1)^3$. As a corollary, since IP_2 on $2n$ variables has spectrum $\hat{f}(S,T) = (-1)^{|S \cap T|} \cdot 2^{-n}$ for every (S,T) and hence $\|\hat{\text{IP}}_2\|_1 = 2^n$, every read-twice oblivious BP for IP_2 has size at least $2^{\lceil n/3 \rceil} - 1$.

Rationale (proposer's reasoning):

The Fourier $\square^1$ norm controls approximate junta degree, sign-rank, communication complexity, and PAC-learnability, yet has barely been linked to branching-program size beyond the read-once width- w bound. Crucially the property $\|\hat{f}\|_1 \leq \text{poly}(s)$ is a SMALLNESS property — random Boolean functions saturate Cauchy-Schwarz with $\|\hat{f}\|_1 \approx 2^{\lceil n/2 \rceil}$ — so it cleanly evades the Razborov-Rudich largeness condition that killed previous tail-mass attempts. The conjecture asserts a structural Bruck-Smolensky-style spectral concentration that read-twice gates inherit from their layered, bounded-history dynamics, while IP_2 's uniform spectrum forces exponential blow-up.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b2e187c3c8db7131

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each (n,w) with $n \in \{6,8,10,12,14\}$, $w \in \{2,3,4\}$, run 30 seeded random read-twice oblivious layered BPs (size $s=2nw+2$). Compute $\|\hat{f}\|_1$ exactly via Walsh-Hadamard. Conjecture

is SUPPORTED iff $\geq 28/30$ seeds satisfy $\|\hat{f}\|_1 \leq (s+1)^3$ at every (n, w) ; FALSIFIED by any single seed with $\|\hat{f}\|_1 > (s+1)^3$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.93	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Fourier L1 norm branching program spectral norm - read-twice oblivious branching program lower bound inner product - spectral norm Fourier sparsity oblivious read-k branching program

Top relevant hits considered: - [<http://arxiv.org/abs/2105.00244v1>] l1-Norm Minimization with Regula Falsi Type Root Finding Methods - [<http://arxiv.org/abs/0904.1710v2>] Comparison of matrix norms on bipartite spaces - [<http://arxiv.org/abs/2603.18511v2>] Norm-trace and Kloosterman sums in finite semi-simple algebras - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/1511.07136v1>] Identity Testing and Lower Bounds for Read- k Oblivious Algebraic Branching Programs - [<http://arxiv.org/abs/1709.08890v2>] Partial matching width and its application to lower bounds for branching programs - [<http://arxiv.org/abs/1708.02054v1>] Pseudorandom Bits for Oblivious Branching Programs - [<http://arxiv.org/abs/1306.3004v2>] Pseudo-randomness for Regular Branching Programs via Fourier Analysis

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def simulate_bp(bp, input_):
        current_state = bp['start']
        for bit in bin(input_)[2:].zfill(len(bp['variable_order'])):
            if bit == '0':
                next_state = bp['transitions'][current_state]['0']
            else:
                next_state = bp['transitions'][current_state]['1']
            current_state = next_state
        return bp['accept_states'].get(current_state, 0)
```

```

def walsh_hadamard_transform(truth_table):
    n = len(truth_table)
    f_hat = [0] * (1 << n)

    for S in range(1 << n):
        sum_val = 0
        for x in range(1 << n):
            sign = (-1) ** bin(S & x).count('1')
            sum_val += sign * truth_table[x]
        f_hat[S] = sum_val / math.sqrt(n)

    return f_hat

def norm_l1(f_hat):
    return sum(abs(val) for val in f_hat)

n_values = [6, 8, 10, 12, 14]
w_values = [2, 3, 4]
instances_tested = 0
total_value = 0

for n in n_values:
    for w in w_values:
        for _ in range(30):
            # Construct the BP
            variable_order = list(range(n))
            random.shuffle(variable_order)
            transitions = {}
            accept_states = set()

            current_state = 0
            for i in range(w):
                new_state = len(transitions) + 1
                transitions[current_state] = {'0': new_state, '1': new_state}
                current_state = new_state

            for _ in range(n - w):
                new_state = len(transitions) + 1
                transitions[current_state] = {'0': new_state, '1': new_state}
                current_state = new_state

            accept_states.add(current_state)

            bp = {
                'variable_order': variable_order,
                'transitions': transitions,
                'accept_states': accept_states,
                'start': 0
            }

            # Simulate the BP and compute the truth table
            truth_table = [simulate_bp(bp, input_) for input_ in range(1 << n)]
            f_hat = walsh_hadamard_transform(truth_table)
            norm_value = norm_l1(f_hat)

            instances_tested += 1

```

```

total_value += norm_value

if norm_value > (2 * w * n + 2 + 1) ** 3:
    return {
        "metric_name": "Fourier L1 Norm",
        "metric_value": norm_value,
        "instances_tested": instances_tested,
        "conjecture_holds": False,
        "counterexample": f"BP of size {2 * w * n + 2} with n={n}, w={w}"
    }

mean_value = total_value / instances_tested
support_fraction = (instances_tested >= 84) # At least 28/30 seeds for each (n, w)

return {
    "metric_name": "Fourier L1 Norm",
    "metric_value": mean_value,
    "instances_tested": instances_tested,
    "conjecture_holds": support_fraction,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res['metric_value'] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res['conjecture_holds']) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=NA support_fraction={support_fraction}")
    elif any(not res['conjecture_holds'] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=NA first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4dc42f6c.py", line 117, in <module>
 result = run_trial(seed)

```

^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4dc42f6c.py", line 82, in run_trial
    truth_table = [simulate_bp(bp, input_) for input_ in range(1 << n)]
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4dc42f6c.py", line 29, in simulate_bp
    return bp['accept_states'].get(current_state, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'set' object has no attribute 'get'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an AttributeError ('set' object has no attribute 'get') in simulate_bp before any trial completed, so no data on $\|\hat{f}\|_1$ vs $(s+1)^3$ was produced. The pre-registered support criterion cannot be evaluated. | next: Fix the BP data structure bug—represent accept_states as a dict (or use membership testing on a set) in simulate_bp—then rerun the full 450-seed sweep across $(n,w) \in \{6,8,10,12,14\} \times \{2,3,4\}$.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	380228
2	preregistration	claude_max	opus	0	0	6059
3	novelty	claude_max	opus	0	0	3448
4	novelty	claude_max	opus	0	0	8666
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12419
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11511
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10971
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12603
9	judge	claude_max	opus	0	0	4856

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 450761 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/90dea97bbe27.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/90dea97bbe27.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/90dea97bbe27.tar.gz` (if generated)